

1.a. Write a C program to insert an element at a particular position of an existing array.

CODE:

```
#include <stdio.h>
int main ( ) {
    int arr [100], n, i, POS, value;
    printf ("Enter the size of the array: ");
    scanf ("%d", &n);
    printf ("Enter array Elements: \n");
    for (i=0; i<n; i++) {
        scanf ("%d", &arr [i]);
    }
    printf ("Enter the position to insert element: ");
    scanf ("%d", &POS);
    printf ("Enter value to insert: ");
    scanf ("%d", &value);
    for (i=n; i>=POS; i--) {
        arr [i] = arr [i-1];
    }
    arr [POS - 1] = value;
    n++;
    printf ("Array after insertion: \n");
    for (i=0; i<n; i++) {
        printf ("%d \n", arr [i]);
    }
    return 0;
}
```

1.b. Write a C program to delete an element at a particular position of an existing array.

CODE:

```
#include<stdio.h>
int main ( ) {
    int arr [100], n, i, pos, value;
    printf ("Enter the size of the array: ");
    scanf ("%d", &n);
    printf ("Enter array Elements: \n");
    for (i=0; i<n; i++) {
        scanf ("%d", &arr [i]);
    }
    printf ("Enter the position of the array you want to delete the element: ");
    scanf ("%d", &pos);
    for (i=pos-1; i<n-1; i++) {
        arr [i] = arr [i+1];
    }
    n--;
    printf ("After deletion array is: \n");
    for (i=0; i<n; i++) {
        printf ("%d \n", arr [i]);
    }
    return 0;}
}
```

1.c. Write a C program to implement a linear search method to find a key in an array.

CODE:

```
#include <stdio.h>
int main ( ) {
```

```

int n, key, i, found = 0;
printf ("Enter number of elements: ");
scanf ("%d", &n);
int arr [n];
printf ("Enter %d elements: \n", n);
for (i=0; i<n; i++) {
    scanf ("%d", &arr [i]);
}
printf ("Enter the key to search: ");
scanf ("%d", &key);
for (i=0; i<n; i++) {
    if (arr [i] == key) {
        printf ("key %d found at position %d\n", key,
            i + 1);
        found = 1;
        break;
    }
}
if (!found) {
    printf ("key %d not found in the array\n", key);
}
return 0;
}

```

1.d. Write a C program to find a key through a binary search method.

CODE:

```

#include <stdio.h>
int main() {
    int arr[50], n, key;
    int low, high, mid;
    int found = 0;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements in sorted order:\n", n);
    // Store array elements
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    // Enter key to search
    printf("Enter element to search: ");
    scanf("%d", &key);
    // Binary Search
    low = 0;
    high = n - 1;
    while (low <= high) {
        mid = (low + high) / 2;
        if (arr[mid] == key) {
            printf("Key found at position %d\n", mid + 1);
            found = 1;
            break;
        }
        else if (arr[mid] < key) {
            low = mid + 1;
        }
    }
}

```

```

    }
    else {
        high = mid - 1;
    }
}
if (!found) {
    printf("Key not found\n");
}
return 0;
}

```

1.e. Write a C program to find a key through a interpolation search method.

CODE:

```

#include <stdio.h>
int interpolationSearch(int arr[], int n, int key)
{
    int low = 0, high = n - 1;
    while (low <= high && key >= arr[low] && key <= arr[high])
    {
        if (arr[low] == arr[high])
        {
            if (arr[low] == key)
                return low;
            else
                return -1;
        }
        int pos = low + ((key - arr[low]) * (high - low)) /
                    (arr[high] - arr[low]);
        if (arr[pos] == key)
            return pos;
        if (arr[pos] < key)
            low = pos + 1;
        else
            high = pos - 1;
    }
    return -1;
}
int main()
{
    int n, key;
    printf ("Enter number of elements: ");
    scanf ("%d", &n);
    int arr [n];
    printf ("Enter %d sorted elements:\n", n);
    for (int i = 0; i < n; i++)
        scanf ("%d", &arr [i]);
    printf ("Enter key to search: ");
    scanf ("%d", &key);
    int result = interpolationSearch (arr, n, key);
    if (result != -1)
        printf ("key found at index %d\n", result);
    else

```

```
    printf ("key not found \n");  
    return 0;  
}
```

2.a. Write a Menu driven C program to implement stack operations.

1.Push

2.Pop

3.Peek

4.Display

CODE:

```
#include <stdio.h>  
#define MAX 5  
int stack [MAX];  
int top = -1;  
void push ( ) {  
    int value;  
    if (top == MAX - 1) {  
        printf ("stack Overflow!\n");  
    } else {  
        printf ("Enter value to push: ");  
        scanf ("%d", &value);  
        top++;  
        stack [top] = value;  
        printf ("value pushed successfully.\n");  
    }  
}  
void pop ( ) {  
    if (top == -1) {  
        printf ("stack Underflow!\n");  
    } else {  
        printf ("popped value: %d\n", stack [top]);  
        top--;  
    }  
}  
void peek ( ) {  
    if (top == -1) {  
        printf ("stack is empty.\n");  
    } else {  
        printf ("Top element: %d\n", stack [top]);  
    }  
}  
void display ( ) {  
    int i;  
    if (top == -1) {  
        printf ("stack is empty.\n");  
    } else {  
        printf ("Stack elements: \n");  
        for (i = top; i >= 0; i--) {  
            printf ("%d\n", stack [i]);  
        }  
    }  
}  
int main ( ) {
```

```

int choice;
while (1) {
    printf ("\n... Stack Operations ....\n");
    printf ("1. Push\n");
    printf ("2. Pop\n");
    printf ("3. Peek\n");
    printf ("4. Display\n");
    printf ("Enter your choice: ");
    scanf ("%d", &choice);
    switch (choice) {
        case 1: push ( ); break;
        case 2: pop ( ); break;
        case 3: peek ( ); break;
        case 4: display ( ); break;
        case 5: return 0;
    }
}
}

```

2.b. Write a program to check whether the parentheses are properly arranged or not e.g. `{{(a+b)}}` is not properly arranged.

CODE:

```

#include <stdio.h>
#include <string.h>
#define MAX 100
char stack [MAX];
int top = -1;
void push (char c) {
    stack [++top] = c;
}
char pop ( ) {
    return stack [top--];
}
int isMatching (char open, char close) {
    if (open == '(' && close == ')') return 1;
    if (open == '{' && close == '}') return 1;
    if (open == '[' && close == ']') return 1;
    return 0;
}
int main ( ) {
    char exp [100];
    int i, flag = 1;
    printf ("Enter expression: ");
    scanf ("%s", exp);
    for (i = 0; i < strlen(exp); i++) {
        if (exp [i] == '(' || exp [i] == '{' || exp [i] == '[')
            push (exp [i]);
        else if (exp [i] == ')' || exp [i] == '}' || exp [i] == ']') {
            if (top == -1 || !isMatching (pop ( ), exp [i])) {
                flag = 0;
                break;
            }
        }
    }
}

```

```

}
if (top != -1) flag = 0;
if (flag)
    printf ("properly arranged\n");
else
    printf ("Not properly arranged\n");
return 0;
}

```

2.c. Write a C program to convert the given infix expression into postfix from using stack.

CODE:

```

#include <stdio.h>
#include <ctype.h>
char s[100];
int top = -1;
int preco(char c)
{
    if (c == '+' || c == '-')
        return 1;
    if (c == '*' || c == '/')
        return 2;
    if (c == '^')
        return 3;
    return 0;
}
int main()
{
    char in[100], post[100], c;
    int i = 0, k = 0;
    printf("Enter infix: ");
    scanf("%s", in);
    while ((c = in[i++]) != '\0')
    {
        if (isalnum(c))
        {
            post[k++] = c;
        }
        else if (c == '(')
        {
            s[++top] = c;
        }
        else if (c == ')')
        {
            while (s[top] != '(')
                post[k++] = s[top--];
            top--;
        }
        else
        {
            while (top != -1 && preco(s[top]) >= preco(c))
                post[k++] = s[top--];
            s[++top] = c;
        }
    }
}

```

```

}
while (top != -1)
    post[k++] = s[top--];
post[k] = '\0';
printf("postfix expression: %s\n", post);
return 0;
}

```

2.d. Write a C program to evaluate the postfix expression using stack.

CODE:

```

#include <stdio.h>
#include <ctype.h>
#include <math.h>
#define MAX 100
int stack [MAX];
int top = -1;
void push (int x) {
    stack [++top] = x;
}
int pop ( ) {
    return stack [top--];
}
int main ( ) {
    char postfix [MAX];
    int value [26];
    int i, op1, op2, result;
    printf ("Enter postfix expression: ");
    scanf ("%s", postfix);
    for (i = 0; i < 26; i++) {
        value [i] = -1;
    }
    for (i = 0; postfix [i] != '\0'; i++) {
        if (isalpha (postfix [i])) {
            int index = postfix [i] - 'a';
            if (value [index] == -1) {
                printf ("Enter value of %c: ", postfix [i]);
                scanf ("%d", &value [index]);
            }
        }
    }
    for (i = 0; postfix [i] != '\0'; i++) {
        if (isalpha (postfix [i])) {
            push (value [postfix [i] - 'a']);
        }
        else {
            op2 = pop ( );
            op1 = pop ( );

            switch (postfix [i]) {
                case '+': result = op1 + op2; break;
                case '-': result = op1 - op2; break;
                case '*': result = op1 * op2; break;
                case '/': result = op1 / op2; break;
            }
        }
    }
}

```

```

        case '^': result = (int) pow (op1, op2); break;
    }
    push (result);
}
}
printf ("Result = %d \n", pop ( ));
return 0;
}

```

2.e. Write a Menu driven C program to implement linear queue operations

1. Insert

2. Delete

3. Display

CODE:

```

#include <stdio.h>
#define MAX 50
int queue [MAX];
int front = -1, rear = -1;
void insert( )
{
    int item;
    if (rear == MAX - 1)
    {
        printf("Overflow!\n");
        return;
    }
    if (front == -1)
        front = 0;
    printf("Enter element to insert: ");
    scanf("%d", &item);
    rear++;
    queue [rear] = item;
    printf("Inserted successfully\n");
}
void delete( )
{
    if (front == -1 || front > rear)
    {
        printf("Underflow!\n");
        return;
    }
    printf("Deleted element: %d\n", queue[front]);
    front++;
    if (front > rear)
    {
        front = rear = -1;
    }
}
void display( )
{
    int i;
    if (front == -1)

```



```

{
    printf("Queue is empty\n");
    return;
}
printf("Queue elements are: \n");
for (i = front; i <= rear; i++)
{
    printf("%d ", queue [i]);
}
printf("\n");
}
int main( )
{
    int ch;
    do
    {
        printf("\nLinear queue operations:\n");
        printf("1. Insert\n");
        printf("2. Delete\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &ch);
        switch (ch)
        {
            case 1:
                insert( );
                break;
            case 2:
                delete( );
                break;
            case 3:
                display( );
                break;
            case 4:
                printf("Exiting program\n");
                break;
            default:
                printf("Wrong input\n");
        }
    } while (ch != 4);
    return 0;
}

```

2.F Write a program to implement the circular queue in the same manner.

1.Enqueue

2.Dequeue

3.Display

CODE:

```

#include <stdio.h>
#include <stdlib.h>
#define MAX 5
int queue[MAX];

```

```

int front = -1, rear = -1;
int isFull(){
    if ((front == 0 && rear == MAX - 1) || (front == rear + 1))
        return 1;
    return 0;
}
int isEmpty(){
    if (front == -1)
        return 1;
    return 0;
}
void enqueue(int value){
    if(isFull()){
        printf("Queue is Full!\n");
        return;
    }
    if (front == -1) {
        front = 0;
    }
    rear=(rear+1)%MAX;
    queue[rear]=value;
    printf("Insert:%d\n",value);
}
void dequeue() {
    if (isEmpty()) {
        printf("Queue is Empty!\n");
        return;
    }
    printf("Deleted element is: %d\n", queue[front]);

    if (front == rear) {
        front = rear = -1;
    } else{
        front=(front+1)% MAX ;
    }
}
void display() {
    if(isEmpty()){
        printf("Queue is empty!\n");
        return;
    }
    int i=front;
    printf("Queue elements : ");
    while (1){
        printf("%d",queue[i]);
        if (i == rear)
            break;
        i=(i+1)%MAX;
    }
    printf("\n");
}
int main() {

```

```

int choice,value;
while(1) {
    printf("\n... Circular Queue Operations ...\n");
    printf("1. Enqueue\n");
    printf("2. Dequeue\n");
    printf("3. Display\n");
    printf("4. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch (choice) {
        case 1:
            printf("Enter value:");
            scanf("%d",&value);
            enqueue(value);
            break;
        case 2: dequeue(); break;
        case 3: display(); break;
        case 4: exit(0);
        default: printf("Invalid choice!\n");
    }
}
return 0;
}

```

3.a. Write a Menu driven C program to Implement following operations on a singly linked list:

- 1.insert at the beginning**
- 2.insert at the end**
- 3.insert at a particular element**
- 4.insert before a particular element**
- 5.delete first node**
- 6.delete last node**
- 7.delete the middle element**
- 8.delete any node (input the element value)**
- 9. display all nodes**
- 10.reverse display off all nodes**
- 11.physical reverse the list**

CODE:

```

#include <stdio.h>
#include <stdlib.h>
struct Node {
    int value;
    struct Node *next;
};
typedef struct Node Node;

```

```
Node *head = NULL;
```

```
// 1. Insert at beginning
```

```

void insert_begin(int x) {
    Node *newnode;
    newnode = (Node*)malloc(sizeof(Node));
    newnode->value = x;
    newnode->next = head;
}

```

```

    head = newnode;
}
// 2. Insert at end
void insert_end(int x) {
    Node *newnode, *current;
    newnode = (Node*)malloc(sizeof(Node));
    newnode->value = x;
    newnode->next = NULL;
    if (head == NULL) {
        head = newnode;
    }
    else {
        current = head;
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = newnode;
    }
}
// 3. Insert after a particular node
void insert_after(int a, int x) {
    Node *curr = head;
    while (curr != NULL) {
        if (curr->value == a)
            break;
        curr = curr->next;
    }
    if (curr == NULL) {
        printf("Element not found!\n");
    }
    else {
        Node *newnode;
        newnode = (Node*)malloc(sizeof(Node));
        newnode->value = x;
        newnode->next = curr->next;
        curr->next = newnode;
    }
}

// 4. Insert before a particular node
void insert_before(int a, int x) {
    Node *prev = NULL;
    Node *curr = head;
    Node *newnode;
    while (curr != NULL) {
        if (curr->value == a)
            break;
        prev = curr;
        curr = curr->next;
    }
    if (curr == NULL) {
        printf("Element not found!\n");
    }

```

```

    return;
}
newnode = (Node*)malloc(sizeof(Node));
newnode->value = x;
newnode->next = curr;
if (curr == head) {
    head = newnode;
}
else {
    prev->next = newnode;
}
}
// 5. Delete first node
void delete_first() {
    Node *ptr;
    if (head == NULL) {
        printf("List is empty!\n");
        return;
    }
    ptr = head;
    head = head->next;
    free(ptr);
}
// 6. Delete last node
void delete_last() {
    Node *curr, *prev;
    if (head == NULL) {
        printf("List is empty!\n");
        return;
    }
    prev = curr = head;
    while (curr->next != NULL) {
        prev = curr;
        curr = curr->next;
    }
    if (prev == curr) {
        head = NULL;
    }
    else {
        prev->next = NULL;
    }
    free(curr);
}
// 7. Delete middle node
void delete_middle() {
    if (head == NULL || head->next == NULL) {
        return;
    }
    Node *slow = head;
    Node *fast = head;
    Node *prev = NULL;
    while (fast != NULL && fast->next != NULL) {

```

```

    fast = fast->next->next;
    prev = slow;
    slow = slow->next;
}
prev->next = slow->next;
free(slow);
}
// 8. Delete node by value
void delete_value(int key) {
    Node *temp = head;
    Node *prev = NULL;
    if (head == NULL) {
        return;
    }
    if (head->value == key) {
        head = head->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->value != key) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Element not found!\n");
        return;
    }
    prev->next = temp->next;
    free(temp);
}

```

```

// 9. Display list
void display() {
    Node *temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->value);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

```

// 10. Reverse display using recursion
void reverse_display(Node *temp) {
    if (temp == NULL)
        return;
    reverse_display(temp->next);
    printf("%d -> ", temp->value);
}

```

```

// 11. Reverse list physically
void reverse_list() {
    Node *prev = NULL;
    Node *current = head;
    Node *next = NULL;

```

```

while (current != NULL) {
    next = current->next;
    current->next = prev;
    prev = current;
    current = next;
}
head = prev;
}

int main() {
    int choice, value, key;
    while (1) {
        printf("\n----- MENU ----- \n");
        printf("1. Insert at beginning \n");
        printf("2. Insert at end \n");
        printf("3. Insert after element \n");
        printf("4. Insert before element \n");
        printf("5. Delete first node \n");
        printf("6. Delete last node \n");
        printf("7. Delete middle node \n");
        printf("8. Delete node by value \n");
        printf("9. Display list \n");
        printf("10. Reverse display \n");
        printf("11. Reverse list physically \n");
        printf("12. Exit \n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                insert_begin(value);
                break;
            case 2:
                printf("Enter value: ");
                scanf("%d", &value);
                insert_end(value);
                break;
            case 3:
                printf("Enter key and value: ");
                scanf("%d %d", &key, &value);
                insert_after(key, value);
                break;
            case 4:
                printf("Enter key and value: ");
                scanf("%d %d", &key, &value);
                insert_before(key, value);
                break;
            case 5:
                delete_first();
                break;
            case 6:
                delete_last();

```

```

        break;
    case 7:
        delete_middle();
        break;
    case 8:
        printf("Enter value to delete: ");
        scanf("%d", &key);
        delete_value(key);
        break;
    case 9:
        display();
        break;
    case 10:
        reverse_display(head);
        printf("NULL\n");
        break;
    case 11:
        reverse_list();
        printf("List reversed successfully.\n");
        break;
    case 12:
        exit(0);
    default:
        printf("Invalid choice!\n");
    }
}
return 0;
}

```

write a C program to Implement basic operations on a doubly linked list: insert at the beginning

CODE:

```

#include <stdio.h> #include
<stdlib.h>
// Structure of node struct Node
{
    int data;
    struct Node *prev; struct Node
    *next;
};
struct Node *head = NULL;
// Function to insert at beginning void
insertBeginning(int value)
{
    struct Node *newNode;
// Allocate memory
    newNode = (struct Node *)malloc(sizeof(struct Node));
// Assign value
    newNode->data = value;
// Set previous as NULL newNode->
prev = NULL;

```



```

// Point next to current head newNode-
>next = head;
// If list is not empty if (head !=
NULL)
{
    head->prev = newNode;
}
// Move head to new node head = newNode;
printf("Node inserted at beginning.\n");
}

// Function to display list
void display()
{
    struct Node *temp;
    if (head == NULL)
    {
        printf("List is empty.\n"); return;
    }

    temp = head;
    printf("Doubly Linked List: "); while (temp !=
    NULL)
    {
        printf("%d ", temp->data); temp = temp-
        >next;
    }
    printf("\n");
}

// Main function int main()
{
    int n, i, value;

    printf("Enter number of nodes: "); scanf("%d", &n);
    for (i = 1; i <= n; i++)
    {
        printf("Enter value %d: ", i); scanf("%d", &value);

        insertBeginning(value);
    }

    display();

    return 0;
}

```

write a C program to Implement basic operations on a doubly linked list: insert at the end

CODE:

```
#include <stdio.h>
#include <stdlib.h>

// Structure of node struct Node
{
    int data;
    struct Node *prev; struct Node
    *next;
};

struct Node *head = NULL;
// Function to insert at end void
insertEnd(int value)
{
    struct Node *newNode, *temp;
    // Allocate memory
    newNode = (struct Node *)malloc(sizeof(struct Node));

    // Assign value
    newNode->data = value; newNode->
next = NULL;
    // If list is empty if (head ==
    NULL
    {
        newNode->prev = NULL; head = newNode;
    }
    else
    {
        temp = head;

        // Move to last node
        while (temp->next != NULL)
        {
            temp = temp->next;
        }

        // Link new node
        temp->next = newNode; newNode->
prev = temp;
    }

    printf("Node inserted at end.\n");
}

// Function to display list void display()
{
```

```

struct Node *temp;
if (head == NULL)
{
    printf("List is empty.\n"); return;
}

temp = head;
printf("Doubly Linked List: "); while (temp !=
NULL)
{
    printf("%d ", temp->data); temp = temp-
>next;
}

printf("\n");
}

// Main function int main()
{
    int n, i, value;

    printf("Enter number of nodes: "); scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        printf("Enter value %d: ", i); scanf("%d", &value);

        insertEnd(value);
    }

    display();
    return 0;
}

```

write a C program to Implement basic operations on a doubly linked list: delete first node CODE:

```

#include <stdio.h>
#include <stdlib.h>
// Structure of node struct Node
{
    int data;
    struct Node *prev; struct Node
    *next;
};

struct Node *head = NULL;
// Function to insert at end void
insertEnd(int value)
{

```

```

struct Node *newNode, *temp;
newNode = (struct Node *)malloc(sizeof(struct Node)); newNode->
data = value;
newNode->next = NULL;
if (head == NULL)
{
    newNode->prev = NULL; head = newNode;
}
else
{
    temp = head;
    while (temp->next != NULL)
    {
        temp = temp->next;
    }
    temp->next = newNode; newNode->
prev = temp;
}
}

// Function to delete first node void
deleteFirst()
{
    struct Node *temp;
    if (head == NULL)
    {
        printf("List is empty.\n"); return;
    }

    temp = head;
    head = head->next; if (head !=
NULL)
    {
        head->prev = NULL;
    }
    free(temp);

    printf("First node deleted.\n");
}

// Function to display list void display()
{
    struct Node *temp;
    if (head == NULL)
    {

```

```

    printf("List is empty.\n"); return;
}

temp = head;
printf("Doubly Linked List: "); while (temp !=
NULL)
{
    printf("%d ", temp->data); temp = temp-
>next;
}

printf("\n");
}

// Main function int main()
{
    int n, i, value;
    printf("Enter number of nodes: "); scanf("%d", &n);
    // User input for nodes for (i = 1; i <=
n; i++)
    {
        printf("Enter value %d: ", i); scanf("%d", &value);

        insertEnd(value);
    }

    printf("\nBefore deletion:\n"); display();
    // Delete first node deleteFirst();
    printf("\nAfter deletion:\n"); display();

    return 0;
}

```

write a C program to Implement basic operations on a doubly linked list: delete last node
CODE:

```

#include <stdio.h>
#include <stdlib.h>
// Structure of node struct Node
{
    int data;
    struct Node *prev; struct Node
    *next;};
struct Node *head = NULL;

// Function to insert at end void
insertEnd(int value)
{
    struct Node *newNode, *temp;
    newNode = (struct Node *)malloc(sizeof(struct Node)); newNode->

```

```

>data = value;
newNode->next = NULL;

if (head == NULL)
{
    newNode->prev = NULL; head = newNode;
}
else
{
    temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->next = newNode; newNode-
    >prev = temp;
}
}

// Function to delete last node void deleteLast()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty.\n"); return;
    }

    temp = head;

    // If only one node
    if (temp->next == NULL)
    {
        head = NULL; free(temp);

        printf("Last node deleted.\n"); return;
    }

    // Move to last node
    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->prev->next = NULL; free(temp);
}

```

```

    printf("Last node deleted.\n");
}

// Function to display list void display()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty.\n"); return;
    }

    temp = head;
    printf("Doubly Linked List: "); while (temp !=
    NULL)
    {
        printf("%d ", temp->data); temp = temp-
        >next;
    }

    printf("\n");
}

// Main function int main(){
    int n, i, value;

    printf("Enter number of nodes: "); scanf("%d", &n);

    // User input
    for (i = 1; i <= n; i++)
    {
        printf("Enter value %d: ", i); scanf("%d", &value);

        insertEnd(value);
    }

    printf("\nBefore deletion:\n"); display();

    // Delete last node deleteLast();

    printf("\nAfter deletion:\n"); display();

    return 0;
}

```

write a C program to Implement basic operations on a doubly linked list: display all nodes CODE:

```
#include <stdio.h> #include
<stdlib.h>

// Structure of node struct Node
{
    int data;
    struct Node *prev; struct Node
    *next;
};

struct Node *head = NULL;

// Function to insert at end void
insertEnd(int value)
{
    struct Node *newNode, *temp;
    newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value; newNode->
    next = NULL;
    if (head == NULL)
    {
        newNode->prev = NULL; head = newNode;
    }
    else
    {
        temp = head;

        while (temp->next != NULL)
        {
            temp = temp->next;
        }

        temp->next = newNode; newNode->
        prev = temp;
    }
}

// Function to display all nodes void display()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty.\n"); return;
    }
}
```



```

}

temp = head;
printf("Doubly Linked List: "); while (temp !=
NULL)
{
    printf("%d ", temp->data); temp = temp-
>next;
}

printf("\n");
}

// Main function int main()
{
    int n, i, value;

    printf("Enter number of nodes: "); scanf("%d", &n);

    // User input for nodes for (i = 1; i <=
n; i++)
    {
        printf("Enter value %d: ", i); scanf("%d", &value);

        insertEnd(value);
    }

    // Display all nodes
    printf("\nAll nodes are:\n"); display();

    return 0;
}

```

Write a C program to Implement the selection sort algorithm. CODE:

```

#include <stdio.h>
int main()
{
    int arr[100], n, i, j, min, temp;
    printf("Enter number of elements: "); scanf("%d",&n);
    printf("Enter elements:\n"); for(i = 0; i < n;
i++)
    {
        scanf("%d", &arr[i]);
    }

    // Selection Sort
    for(i = 0; i < n - 1; i++)
    {

```

```

    min = i;
    for(j = i + 1; j < n; j++)
    {
        if(arr[j] < arr[min])
        {
            min = j;
        }
    }
    temp = arr[i]; arr[i] = arr[min];
    arr[min] = temp;
}
printf("Sorted array:\n"); for(i = 0; i < n;
i++)
{
    printf("%d ", arr[i]);
}
return 0;
}

```

Write a C program to Implement the bubble sort algorithm.

CODE:

```

#include <stdio.h>

int main()
{
    int arr[100], n, i, j, temp;
    printf("Enter number of elements: "); scanf("%d",
    &n);
    printf("Enter elements:\n"); for(i = 0; i < n;
    i++)
    {
        scanf("%d", &arr[i]);
    }

    // Bubble Sort
    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1]; arr[j + 1] =
                temp;
            }
        }
    }
}

```

```

printf("Sorted array:\n"); for(i = 0; i < n;
i++)
{
    printf("%d ", arr[i]);
}

return 0;
}

```

Write a C program to Implement the insertion sort algorithm. CODE:

```

#include <stdio.h>
int main()
{
    int arr[100], n, i, j, key;
    printf("Enter number of elements: "); scanf("%d",
    &n);
    printf("Enter elements:\n"); for(i = 0; i < n;
    i++)
    {
        scanf("%d", &arr[i]);
    }

    // Insertion Sort for(i = 1; i < n;
    i++)
    {
        key = arr[i]; j = i - 1;

        while(j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j]; j--;
        }

        arr[j + 1] = key;
    }

    printf("Sorted array:\n"); for(i = 0; i < n;
    i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}

```

Write a C program to Implement the Quick sort algorithm. CODE:

```

#include <stdio.h>
int partition(int arr[], int low, int high)

```

```

{
    int pivot, i, j, temp;
    pivot = arr[high]; i = low - 1;

    for(j = low; j < high; j++)
    {
        if(arr[j] < pivot)
        {
            i++;
            temp = arr[i]; arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    temp = arr[i + 1]; arr[i + 1] =
    arr[high]; arr[high] = temp;
    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    int pi;

    if(low < high)
    {
        pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1); quickSort(arr, pi
        + 1, high);
    }
}

int main()
{
    int arr[100], n, i;

    printf("Enter number of elements: "); scanf("%d",
    &n);
    printf("Enter elements:\n"); for(i = 0; i < n;
    i++)
    {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1); printf("Sorted
    array:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
}

```

```
}  
return 0;  
}
```

Write a C program to implement the Merge sort algorithm

CODE:

```
#include <stdio.h>  
  
void merge(int arr[], int l, int m, int r)  
{  
    int i, j, k;  
    int n1 = m - l + 1; int n2 = r - m;  
  
    int L[100], R[100];  
    for(i = 0; i < n1; i++) L[i] = arr[l + i];  
  
    for(j = 0; j < n2; j++)  
        R[j] = arr[m + 1 + j];  
    i = 0;  
    j = 0;  
    k = l;  
  
    while(i < n1 && j < n2)  
    {  
        if(L[i] <= R[j])  
        {  
            arr[k] = L[i]; i++;  
        }  
        else  
        {  
            arr[k] = R[j]; j++;  
        }  
        k++;  
    }  
  
    while(i < n1)  
    {  
        arr[k] = L[i]; i++;  
        k++;  
    }  
  
    while(j < n2)  
    {  
        arr[k] = R[j]; j++;  
        k++;  
    }  
}
```

```

void mergeSort(int arr[], int l, int r)
{
    int m;

    if(l < r)
    {
        m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        merge(arr, l, m, r);
    }
}

int main()
{
    int arr[100], n, i;

    printf("Enter number of elements: "); scanf("%d",
    &n);
    printf("Enter elements:\n"); for(i = 0; i < n;
    i++)
    {
        scanf("%d", &arr[i]);
    }

    mergeSort(arr, 0, n - 1);
    printf("Sorted array:\n"); for(i = 0; i < n;
    i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}

```

Write a C program to mplement the Heap sort algorithm

CODE:

```

#include <stdio.h>

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1; int right = 2 * i +
    2; int temp;

    if(left < n && arr[left] > arr[largest]) largest = left;
    if(right < n && arr[right] > arr[largest]) largest = right;

```

```

    if(largest != i)
    {
        temp = arr[i];
        arr[i] = arr[largest]; arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    int i, temp;
    for(i = n / 2 - 1; i >= 0; i--)
    {
        heapify(arr, n, i);
    }

    for(i = n - 1; i > 0; i--)
    {
        temp = arr[0]; arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}

int main()
{
    int arr[100], n, i;

    printf("Enter number of elements: "); scanf("%d",
    &n);
    printf("Enter elements:\n"); for(i = 0; i < n;
    i++)
    {
        scanf("%d", &arr[i]);
    }

    heapSort(arr, n);
    printf("Sorted array:\n"); for(i = 0; i < n;
    i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}

```

Write a Menu driven C program to implement the following operations Binary search tree.

1. Insert a node CODE:

```
#include <stdio.h> #include
<stdlib.h>

// Structure of BST node struct Node
{
    int data;
    struct Node *left; struct Node
    *right;
};

// Function to create node
struct Node* createNode(int value)
{
    struct Node *newNode;
    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value; newNode->
    left = NULL; newNode->right = NULL;

    return newNode;
}

// Function to insert node
struct Node* insert(struct Node *root, int value)
{
    if(root == NULL)
    {
        return createNode(value);
    }

    if(value < root->data)
    {
        root->left = insert(root->left, value);
    }

    else
    {
        root->right = insert(root->right, value);
    }

    return root;
}

// Function to display inorder traversal void
inorder(struct Node *root)
{

```



```

    if(root != NULL)
    {
        inorder(root->left); printf("%d ", root-
        >data); inorder(root->right);
    }
}

// Main function int main()
{
    struct Node *root = NULL; int n, i, value;
    printf("Enter number of nodes: "); scanf("%d", &n);

    // User input for insertion for(i = 1; i <= n;
    i++)
    {
        printf("Enter value %d: ", i); scanf("%d", &value);
        root = insert(root, value);
    }

    printf("\nNodes inserted successfully.\n");
    printf("Inorder Traversal: ");
    inorder(root);
    return 0;
}

```

Write a Menu driven C program to implement the following operations Binary search tree.
preorder traversal of nodes
CODE:

```

#include <stdio.h>
#include <stdlib.h>
// Structure of BST node struct Node
{
    int data;
    struct Node *left; struct Node
    *right;
};

// Function to create node
struct Node* createNode(int value)
{
    struct Node *newNode;
    newNode = (struct Node*)malloc(sizeof(struct Node)); newNode-
    >data = value;
    newNode->left = NULL;
    newNode->right = NULL;
}

```

```

    return newNode;
}

// Function to insert node
struct Node* insert(struct Node *root, int value)
{
    if(root == NULL)
    {
        return createNode(value);
    }

    if(value < root->data)
    {
        root->left = insert(root->left, value);
    }

    else
    {
        root->right = insert(root->right, value);
    }

    return root;
}

// Preorder Traversal
void preorder(struct Node *root)
{
    if(root != NULL)
    {
        printf("%d ", root->data);
        preorder(root->left); preorder(root->right);
    }
}

// Main function int main()
{
    struct Node *root = NULL; int n, i, value;
    printf("Enter number of nodes: "); scanf("%d", &n);

    // User input for insertion for(i = 1; i <= n; i++)
    {
        printf("Enter value %d: ", i); scanf("%d", &value);

        root = insert(root, value);
    }

    printf("\nPreorder Traversal: "); preorder(root);
}

```

```
    return 0;
}
```

**Write a Menu driven C program to implement the following operations Binary search tree.
inorder traversal of nodes**

CODE:

```
#include <stdio.h>
#include <stdlib.h>
// Structure of BST node struct Node
{
    int data;
    struct Node *left; struct Node
    *right;
};

// Function to create node
struct Node* createNode(int value)
{
    struct Node *newNode;
    newNode = (struct Node*)malloc(sizeof(struct Node)); newNode-
    >data = value;
    newNode->left = NULL; newNode-
    >right = NULL;
    return newNode;
}

// Function to insert node
struct Node* insert(struct Node *root, int value)
{
    if(root == NULL)
    {
        return createNode(value);
    }

    if(value < root->data)
    {
        root->left = insert(root->left, value);
    }
    else
    {
        root->right = insert(root->right, value);
    }

    return root;
}

// Inorder Traversal
void inorder(struct Node *root)
```

```

{
    if(root != NULL)
    {
        inorder(root->left); printf("%d ", root-
        >data); inorder(root->right);
    }
}

// Main function int main()
{
    struct Node *root = NULL; int n, i, value;
    printf("Enter number of nodes: "); scanf("%d", &n);

    // User input for insertion for(i = 1; i <= n;
    i++)
    {
        printf("Enter value %d: ", i); scanf("%d", &value);
        root = insert(root, value);
    }

    printf("\nInorder Traversal: ");
    inorder(root);

    return 0;
}

```

Write a Menu driven C program to implement the following operations Binary search tree. postorder traversal of nodes

CODE:

```

#include <stdio.h>
#include <stdlib.h>

// Structure of BST node struct Node
{
    int data;
    struct Node *left; struct Node
    *right;
};

// Function to create node
struct Node* createNode(int value)
{
    struct Node *newNode;
    newNode = (struct Node*)malloc(sizeof(struct Node)); newNode-
    >data = value;
    newNode->left = NULL; newNode->
    right = NULL;
    return newNode;
}

```

```

}

// Function to insert node
struct Node* insert(struct Node *root, int value)
{
    if(root == NULL)
    {
        return createNode(value);
    }

    if(value < root->data)
    {
        root->left = insert(root->left, value);
    }

    else
    {
        root->right = insert(root->right, value);
    }

    return root;
}

// Postorder Traversal
void postorder(struct Node *root)
{
    if(root != NULL)
    {
        postorder(root->left); postorder(root->right); printf("%d ", root->data);
    }
}

// Main function int main()
{
    struct Node *root = NULL; int n, i, value;
    printf("Enter number of nodes: "); scanf("%d", &n);

    // User input for insertion for(i = 1; i <= n; i++)
    {
        printf("Enter value %d: ", i); scanf("%d", &value);

        root = insert(root, value);
    }

    printf("\nPostorder Traversal: "); postorder(root);
    return 0;
}

```